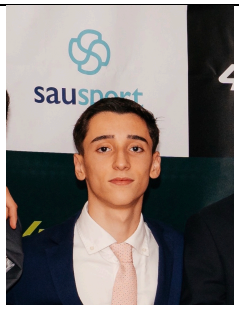
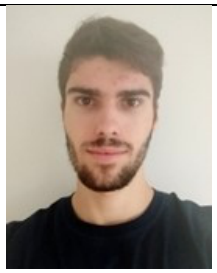
Engenharia Informática

UC Integração na Empresa

Constituição da Equipa

	Nº 22423 Mátio Macedo
	Nº 31419 Guilherme Barros
	Nº 33366 Gonçalo Rodrigues

	Nº 36701 Gonçalo Costa
	Nº 31434 Mário Ferreira

3º ano – 1º semestre

Docente Luís Barreiros

Índice de Figuras

Figura 1 - Arquitetura da Solução 8
Figura 2 - Visão geral do processo 13
Figura 3 - Receção da Reparação 15
Figura 4 - Receção da Montagem 16
Figura 5 - Reparação do Computador 16
Figura 6 - Montagem do Computador 17
Figura 7 - Lista de produtos 22
Figura 8 - Categorias de produtos 22
Figura 9 - Stock por produto 23
Figura 10 - Fatura rascunho 23
Figura 11 - Consulta me 25
Figura 12 - Consulta autenticada de produtos 34
Figura 13 - Criação de uma fatura rascunho 35
Figura 14 - Consulta da fatura 35
Figura 15 - Diagrama do workflow 39
Figura 16 - HTTP request 41
Figura 17 - if 41
Figura 18 - Mensagem de alerta 42
Figura 19 - Email de alerta 42
Figura 20 - Workflow final 43
Figura 21 - Execução lançada pelo agendamento 43
Figura 22 - Diagrama Gantt 51

Índice de Tabelas

Tabela 1 - Endpoints REST 10
Tabela 2 - Etapas do processo 14
Tabela 3 - Entidades do modelo de domínio e mapeamento 20
Tabela 4 - Características da API GraphQL 24
Tabela 5 - Comparação métodos 24
Tabela 6 - Operações da API do Moloni ON 26
Tabela 7 - Classificação de erros 31
Tabela 8 - Operações GraphQL 33
Tabela 9 - Operações pretendidas 37
Tabela 10 - Comparação das ferramentas 39
Tabela 11 - Descrição do Workflow 40
Tabela 12 - Grupo e áreas de responsabilidade 50

Sumário

1. Introdução *6*

1.1 Contextualização **6**

1.2 Objetivos **7**

2. Processo de negócio *8*

2.1 Diagrama Arquitetura **8**

2.2 Descrição detalhada **12**

2.2.1 Intervenientes no processo 12
2.2.2 Visão geral das etapas 13
2.2.3 Diagramas BPMN 15
2.2.4 Integração do agente RPA 19
2.2.5 Modelo de domínio e mapeamento com o Moloni ON 20

3. ERPs *21*

3.1 Moloni ON **21**

Descrição e utilização 21
API 24
Mais-valia e contributo para o processo de negócio 26
Limitações 27

3.2 Jasmin **28**

API 28
Comparação com o Moloni ON 28
Limitações 29

4. Processo de integração *30*

4.1 Moloni ON **30**

4.1.1 Arquitetura e tecnologias utilizadas 30
4.1.2 Segurança e autenticação 31
4.1.3 Endpoints REST implementados 32
4.1.4 Testes realizados 34
4.1.5 Limitações 36

4.2 Jasmin **36**

4.2.1 Abordagem tecnológica 36
4.2.2 Decisão tecnológica 37

5. Automação com n8n *38*

5.1 Decisão tecnológica 38
5.2 Escolha da ferramenta 38
5.3 Descrição do workflow 39
5.4 Testes 41
5.5 Limitações e trabalho futuro 44

6. Implementação *45*
6.1 Modelação do processo **45**
6.2 Modelo de dados **45**
6.3 Interfaces com o utilizador/Formulários **45**
6.4 Regras de negócio **45**
6.5 Performers **45**
6.6 Integração **45**
6.7 Execução do projeto **45**
7. Conclusões *46*
7.1 Objetivos **46**
7.2 Resultados alcançados **46**
7.3 Dificuldades e limitações **46**
8. Referências *47*
9. Anexos *50*
9.1 Gestão de Projeto **50**
Equipa e Responsabilidades 50
Diagrama de Gantt 51
Ferramentas de comunicação inter-equipa 51
Metodologia colaborativa de desenvolvimento 52
Diário do projeto 52
Reuniões de equipa 62

1. Introdução

1.1 Contextualização

O presente relatório documenta o projeto inserido no âmbito da Unidade Curricular de Integração da Empresa, do terceiro ano da Licenciatura em Engenharia Informática da Escola Superior de Tecnologia e Gestão da Universidade Politécnica de Viana do Castelo. Esta Unidade Curricular visa aproximar os estudantes da realidade das organizações, através da análise de um processo de negócio real ou verosímil e da sua automatização com recurso a sistemas de informação empresariais, plataforma de desenvolvimento low-code e ferramentas de automação de processos.

O projeto intitulado "Gestão Inteligente de Assistência Técnica e Intervenções", incide sobre uma pequena empresa de serviços técnicos que presta dois tipos de serviço: a reparação e a montagem de computadores. Nesta organização, o cliente entrega o seu equipamento ou apresenta os requisitos de montagem, é elaborado um orçamento, a intervenção é executada e no final de tudo é emitida a fatura, recebido o pagamento e entregue o equipamento ao cliente.

Criamos um cenário inicial da empresa, onde os registos de pedidos, controlo do stock de peças e a emissão de faturas eram feitas de forma pouco integrada: informação introduzida mais do que uma vez em sítios diferentes, controlo do stock depende de alguém se lembrar de verificar. Isto tudo pode levar a atrasos na intervenção e na entrega ao cliente. Estas limitações são típicas de empresas pequenas, onde a ausência de integração entre a gestão operacional e o sistema de faturação cria riscos de erro e aumenta o trabalho manual repetitivo.

Assim, o grupo, para resolver este problema, propõe uma solução assente em três grandes ideias: utilizar uma plataforma low-code, Mendix, para suportar o processo operacional; esta plataforma permite o desenvolvimento de aplicações empresariais com menor esforço e maior eficiência [1]. A segunda ideia é a ligação a um ERP, neste caso o Moloni ON, responsável pela faturação certificada, pelos clientes e stock de peças, através de uma camada de serviços própria. Por fim, a terceira, é a automatização de tarefas repetitivas com um RPA, n8n, como

a monitorização diária do stock e o envio de um alerta quando uma peça fica abaixo do mínimo. Foi analisado também um segundo ERP, o Jasmin, porém este não foi selecionado.

Este relatório, encontra-se dividido em nos seguintes capítulos: processo de negócio, ERPs, processo de integração, automação com n8n, implementação e conclusões.

1.2 Objetivos

O objetivo geral deste projeto é demonstrar a viabilidade técnica e o benefício de integrar uma aplicação low-code de gestão e assistência técnica com um ERP e com uma automação de processos. Para alcançar esse mesmo objetivo, foram ainda definidos os seguintes objetivos específicos:

- Modelar e descrever o processo de negócio;
- Definir a arquitetura da solução e as responsabilidades de cada componente;
- Analisar e comparar soluções de ERP;
- Implementar uma camada de serviços REST;
- Automatizar com n8n;
- Organizar o trabalho colaborativo e o registo de esforço da equipa.

2. Processo de negócio

2.1 Diagrama Arquitetura

A arquitetura proposta para o sistema de Gestão Inteligente de Assistência Técnica e Intervenções assenta numa abordagem modular onde cada tecnologia tem as suas responsabilidades bem definidas. A solução combina a plataforma low-code Mendix, responsável pela gestão dos processos de assistência técnica, uma camada intermédia de serviços desenvolvida em Node.js e Express, o ERP Moloni ON, para a gestão comercial e administrativa e o n8n para a automatização de tarefas periódicas. Na Figura 1 é possível verificar isto mesmo, estando apresentada a arquitetura lógica, identificando os principais componentes, as responsabilidades e os mecanismos de comunicação.

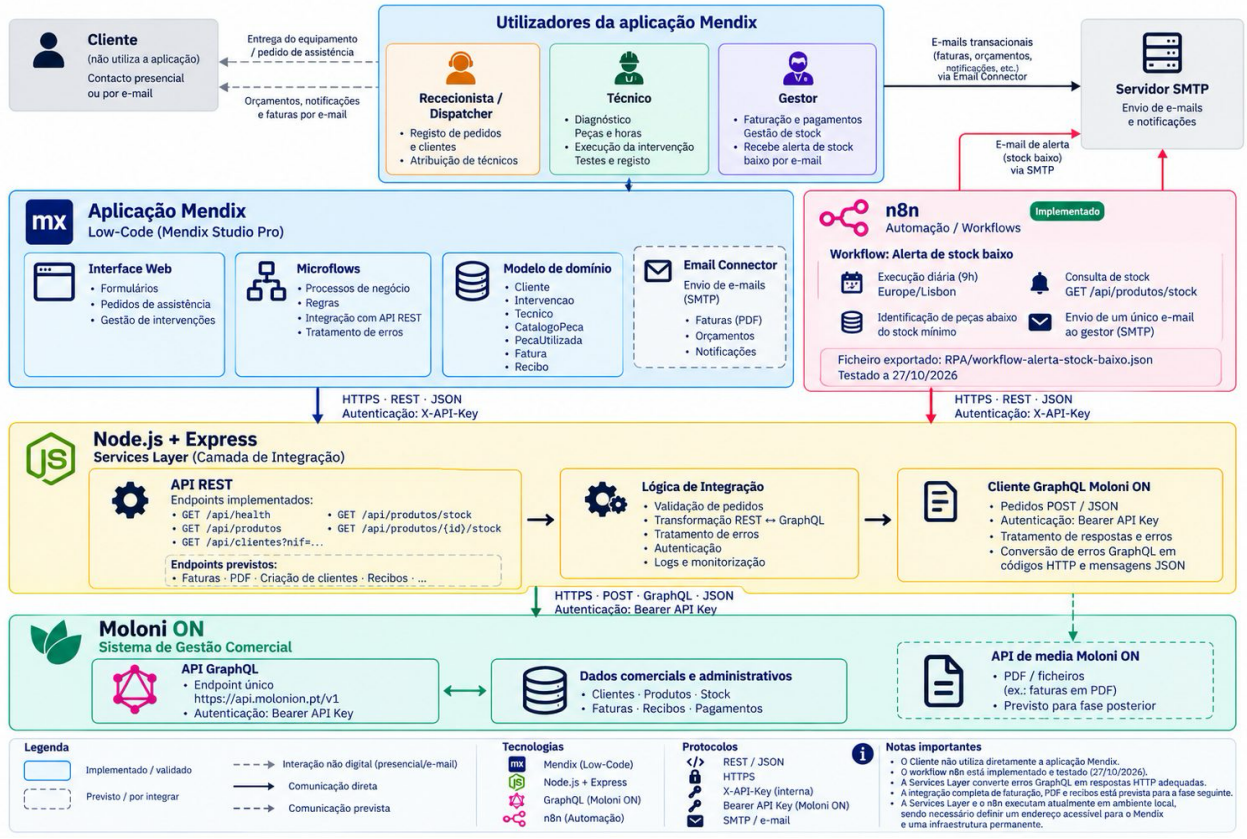


Figura 1 - Arquitetura da Solução

Plataforma Mendix

O Mendix constitui a plataforma central de gestão de processos, abrangendo as várias etapas desde a receção do pedido até ao pagamento e entrega do equipamento. Segundo a Mendix, o Team Server serve para apoiar o trabalho colaborativo e o controlo de versões do projeto [3].
A utilização da aplicação está organizada em três perfis:

- Rececionista: registo inicial dos pedidos e dos clientes, organização e atribuição das intervenções e atribuição de técnicos;
- Técnico: diagnóstico, elaboração de orçamentos, identificação de peças necessárias e realização de testes;
- Gestor: acompanhamento das intervenções, da faturação e dos pagamentos e gestão das necessidades de reposição de stock.

O cliente é um interveniente externo e não utiliza a aplicação, contactando a empresa presencialmente, por telefone ou email.
O modelo de domínio contempla as entidades Cliente, Intervencao, Tecnico, CatalogoPeca, PecaUtilizada, Fatura e Recibo. A entidade Intervencao representa o próprio pedido de assistência, acompanhando-o do registo inicial à conclusão sem a necessidade de uma entidade de Pedido independente. Os processos e regras de negócio são implementadas em microflows que coordenam as etapas e comunicam com os serviços externos.
Para consumir a camada de serviços o Mendix, utiliza o contrato services-layer/openapi.yaml como referência para configurar os serviços REST consumidos [4]. As operações são invocadas através das ações Call Rest Service [5] com o cabecalho X-API-key para autenticação. As respostas JSON são transformadas através de import mappings e de entidades não persistentes, sendo depois transferidos para as entidades persistentes, onde os dados relevantes e as referências ao registo do ERP.

Camada de Integração

Entre o Mendix e o Moloni ON foi desenvolvida uma camada intermédia em Node.js e Express [6]. Esta componente disponibiliza uma API REST que recebe os pedidos dos sistemas, valida parâmetros e credenciais, executa operações sobre a API GraphQL do Moloni ON e transforma resultados em respostas JSON. A camada centraliza autenticação e tratamento de erros, evitando que o Mendix e o n8n tenham de implementar diretamente as operações GraphQL do ERP. Neste momento, estão em implementados e testados 5 endpoints REST que estão listados na Tabela 1.

Tabela 1 - Endpoints REST

Método	Endpoint	Funcionalidade
GET	/api/health	Verificação do estado da camada de serviços
GET	api/produtos	Consulta dos produtos existentes no ERP
GET	api/clientes?nif=...	Pesquisa de clientes através do NIF
GET	api/produtos/stock	Consulta dos produtos com controlo de stock
GET	api/produtos/{id}/stock	Consulta do stock de um produto específico

Foi ainda realizada uma simulação de faturação, através de um script de teste, onde se criou e consultou uma fatura de rascunho no Moloni ON. A implementação dos restantes endpoints REST está previsto para a fase de implementação.

ERP Moloni ON

O Moloni ON é um sistema de gestão comercial e administrativa que centraliza clientes, produtos, stocks, documentos de faturação e pagamentos. A comunicação faz-se através da sua API GraphQL [7], com pedidos HTTPS POST [8]. As consultas e alterações são efetuadas, respectivamente, através de queries e mutations. Esta arquitetura contempla ainda a API de media do Moloni ON para a obtenção de faturas em PDF. Assim, o ERP funciona como sistema de referência para dados comerciais enquanto a informação operacional, como pedidos,

técnicos e intervenções é gerido no Mendix. Foi ainda considerado o Jasmin como alternativa de ERP, mas este acabou por não ser selecionado.

Automação com n8n

A ferramenta N8N automatiza a monitorização das existências. O workflow, "alerta de stock baixo", está implementado e testado, este é executado diariamente às 9:00 fazendo uma consulta get e enviando por SMTP um e-mail ao gestor com as peças cujo stock atual é inferior ao mínimo.

Comunicação e segurança

Em termos de comunicação e segurança, a arquitetura prevê HTTPS entre todos os seus componentes. O Mendix e o n8n comunicam com a camada de serviços por REST ou JSON com uma chave interna enviada. Esta separação concentra assim as credenciais do ERP num único componente. Neste momento, existem dois mecanismos de envio e-mail. O n8n usa o SMTP para alertas de stock e o Mendix está previsto de utilizar também o módulo "e-mail connector" [9] para enviar notificações e faturas em PDF.

2.2 Descrição detalhada

Este processo abrange a prestação de serviços de reparação e montagem de computadores e estrutura-se em oito etapas coordenadas entre seis intervenientes. Os diagramas do processo foram modelados em BPMN [10] e serão apresentados nesta secção.

2.2.1 Intervenientes no processo

Foram definidos como intervenientes no processo: o cliente, rececionista, técnico, gestor, sistema (Mendix + camada de serviços) e o agente RPA. Assim, o cliente é a entidade final que solicita assistência, aprova orçamentos, recebe notificações por e-mail e liquida as faturas. A rececionista faz o acolhimento ao balcão, identificação e registo da ficha do cliente, registo das anomalias ou das especificações de montagem e atribuição das tarefas aos técnicos. Por sua vez, o técnico realiza um diagnóstico e elaboração de orçamentos, verificação e reserva de stock de componentes, execução da reparação ou montagem, testes de controlo de qualidade e registo dos tempos de mão de obra e das peças consumidas. O gestor faz de toda a gestão de negócio, acompanhando as faturas emitidas e os pagamentos, recebendo por e-mail alerta de stock baixo e decidir se as encomendas de peças são feitas ou não ao fornecedor. Já o sistema é a plataforma central que orquestra os microflows, gera os estados de ordem de serviço, comunica por REST com a camada intermédia e despoleta os e-mails transacionais. Por fim, o agente RPA trata do workflow n8n executado diariamente de forma automática. Ele lê o stock das peças através da camada de serviços e envia ao gestor um alerta único com as peças abaixo do stock mínimo.

2.2.2 Visão geral das etapas

Observando a Figura 2, é possível ter uma visão geral do processo, em que está apresentada a sequência das oito etapas, assinalando ainda as que interagem com o ERP. As etapas 1, 3 e 4 são geridas exclusivamente no Mendix, as restantes recorrem ao Moloni ON através da camada de serviço.

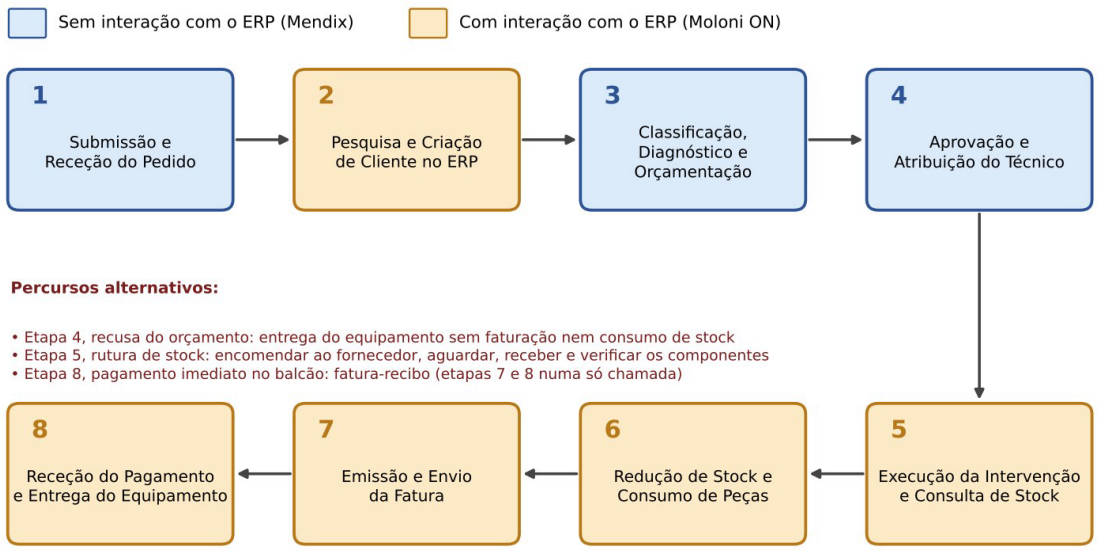


Figura 2 - Visão geral do processo

Assim, o fluxo passa por uma submissão e registo de pedido, pesquisa ou criação do cliente, classificação, diagnóstico e orçamento, aprovação e atribuição do técnico, execução da intervenção e consulta de stock, redução de stock e consumo de peças, emissão e envio de fatura, receção de pagamento e entrega do equipamento.

Existem ainda alguns percursos alternativos, nomeadamente na recusa de orçamento, em que é realizada a entrega sem faturação nem consumo de stock ou na eventualidade uma ruptura de stock, encomendar ao fornecedor. Observando a Tabela 2, é possível reparar nestas mesmas etapas.

Tabela 2 - Etapas do processo

Etapa	Descrição	Interação com o ERP
1. Submissão e receção do pedido	O cliente entrega o computador ou apresenta os requisitos de montagem. A rececionista regista a informação inicial.	Nenhuma
2. Pesquisa e criação do cliente no ERP	Verifica-se pelo NIF do cliente se já existe. Se não, recolhe-se a informação e cria-se uma ficha.	customers, customerNextNumber, customerCreate
3. Classificação, diagnóstico e orçamentação	Diagnóstico, estimativa de custos e notificação do cliente por e-mail.	Nenhuma
4. Aprovação e atribuição do técnico	O cliente aprova o orçamento: a intervenção é atribuída a um técnico disponível.	Nenhuma
5. Execução da intervenção e consulta de stock	O técnico executa a assistência ou montagem e verifica a disponibilidade das peças.	products, product
6. Redução de stock e consumo de peças	Registo das peças consumidas e das horas de mão de obra, após os testes serem bem-sucedidos.	invoiceCreate
7. Emissão e envio da fatura	Consolidação das linhas de mão de obra e peças, emissão do documento	invoiceCreate, invoiceGetPDF, invoiceGetPDFToken

	certificado e envio ao cliente.	
8. Receção do pagamento e entrega do equipamento	Liquidação da fatura, emissão do recibo, entrega do equipamento e encerramento.	receiptCreate, invoice, getCustomerRelatedDocument

2.2.3 Diagramas BPMN

As etapas, descritas anteriormente, estão mapeadas nos quatro diagramas BPMN: na Figura 3, temos a receção da reparação, na Figura 4, a reparação do computador, na figura 5, a receção da montagem e, por fim, na Figura 6 temos a montagem do computador.

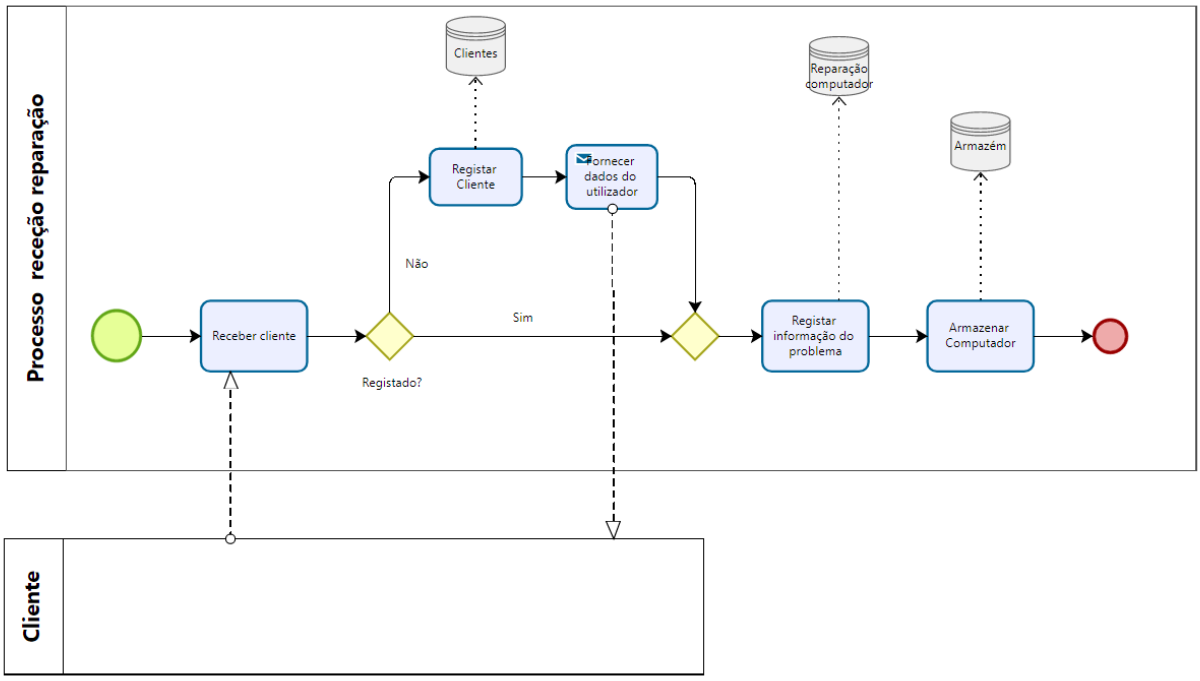


Figura 3 - Receção da Reparação

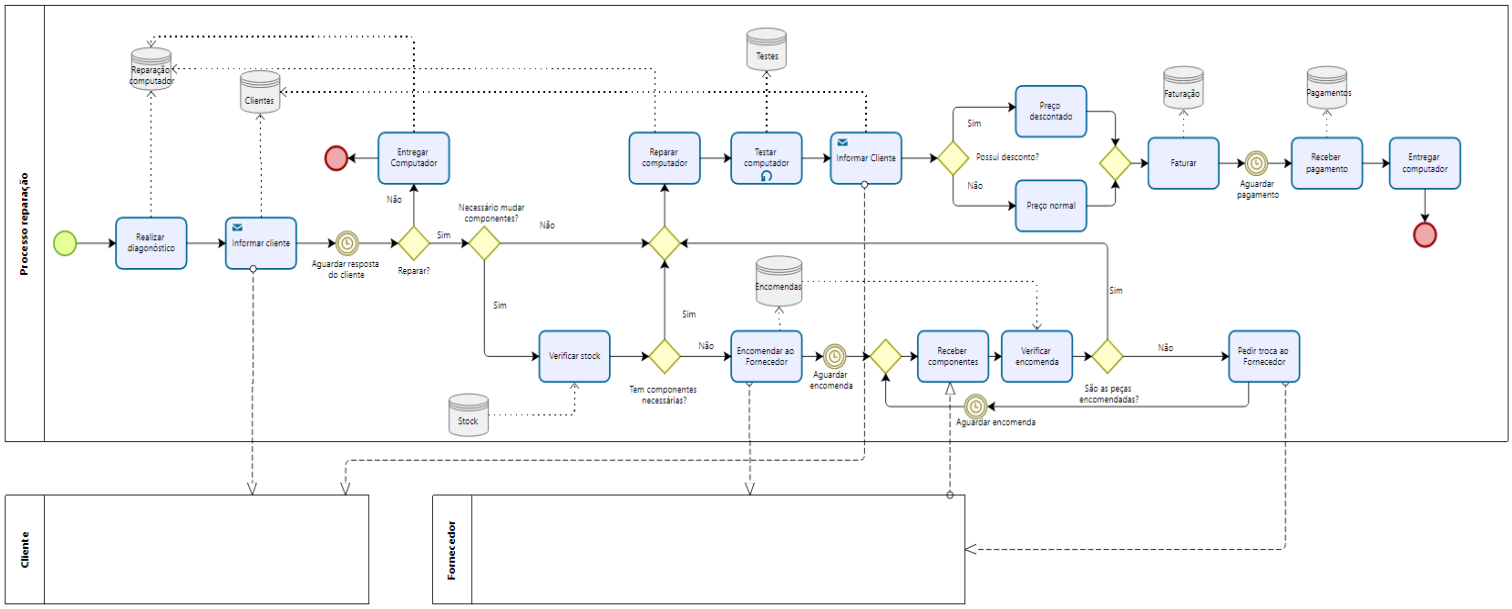


Figura 4 - Reparação do Computador

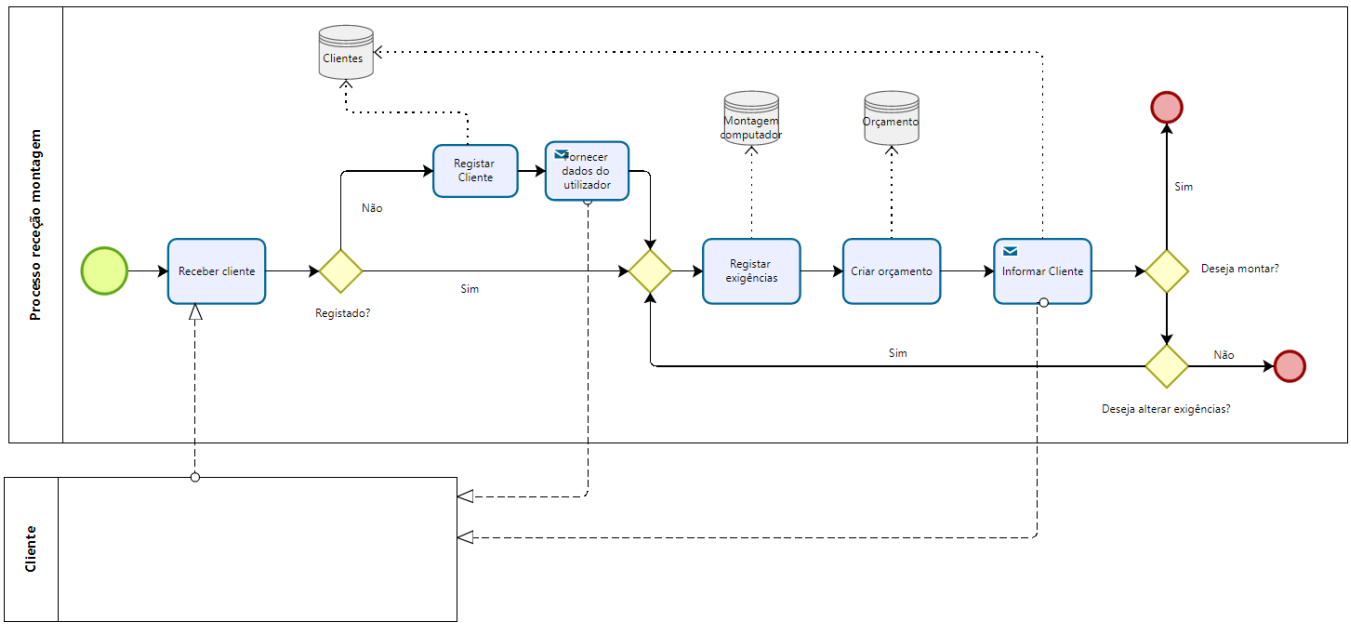


Figura 5 - Receção da Montagem

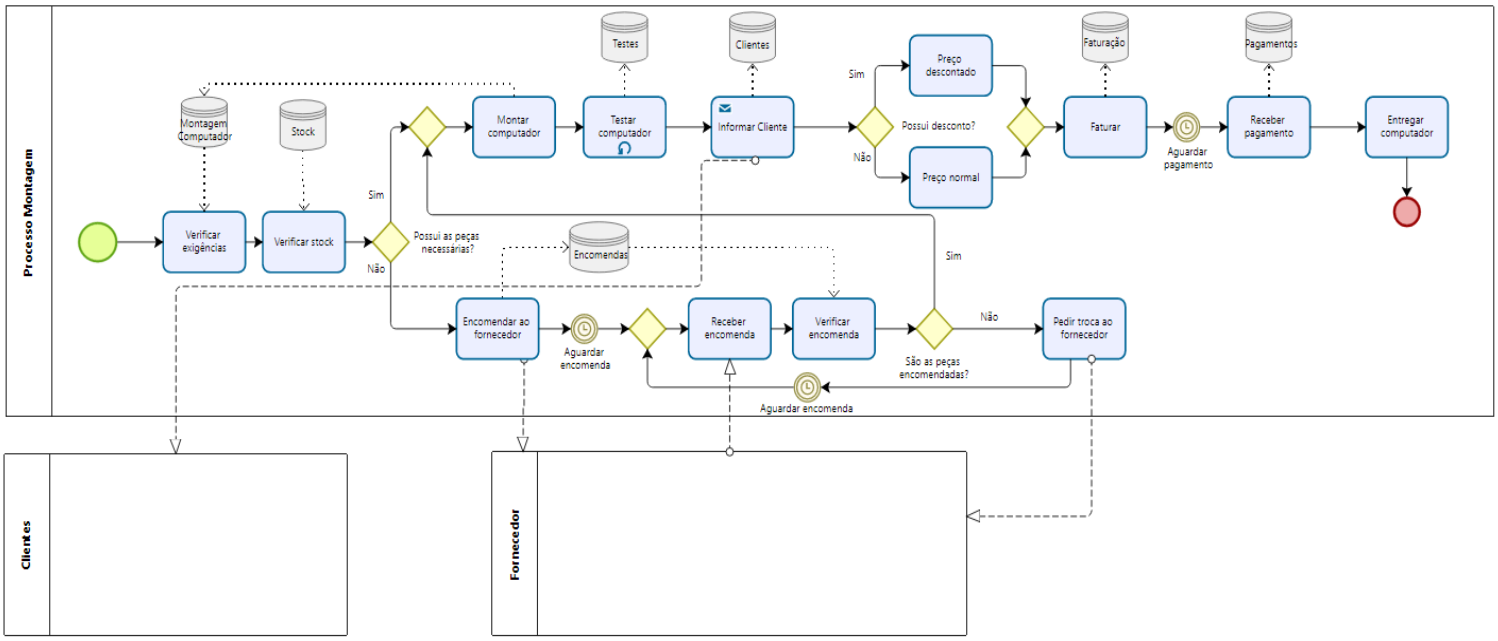


Figura 4 - Montagem do Computador

Submissão e receção do pedido

O cliente entrega o computador ou apresenta os requisitos de montagem e a rececionista regista a informação inicial. No BPMN correspondem as atividades receber cliente, registar informação do problema, armazenar computador, receber cliente e registar exigências.

Pesquisa e criação do cliente no ERP

O sistema valida através do NIF se o cliente já está registado. Se existir, recupera os seus dados. Caso contrário, é recolhida a informação do cliente e a ficha é criada. No BPMN, isto corresponde à decisão registado e à atividade de registar cliente.

Classificação, diagnóstico e orçamentação

No caso das reparações, o técnico inspeciona o equipamento e documenta o diagnóstico. Já no caso das montagens, escolhem-se os componentes compatíveis e calcula-se a estimativa de custos. No fim, o cliente é notificado por e-mail com a proposta. No BPMN, na Figura 4, isto corresponde a realizar diagnóstico e informar cliente. E na Figura 5, corresponde criar orçamento e informar cliente.

Aprovação e atribuição do técnico

Nesta etapa, o cliente avalia a proposta. Se houver aprovação, a intervenção transita para o estado correspondente e é atribuída formalmente a um técnico. No BPMN, isto corresponde às decisões Reparar e Deseja Montar.

Execução da intervenção e consulta de stock

Aqui, o técnico executa a assistência ou a montagem e verifica a disponibilidade das peças no stock. Isto, no BPMN corresponde a verificar stock na reparação e na montagem do computador.

Redução de stock e consumo de peças

Concluída a intervenção e validados os testes, registam-se as peças consumidas e as horas de mão de obra. No BPMN, isto corresponde à finalização técnica e à transição para a faturação.

Emissão e envio da fatura

Na penúltima etapa, o sistema consolida as linhas de mão de obra e de peças aplicadas. Verifica ainda a existência de descontos do cliente, gera o documento fiscal e envia o mesmo ao cliente. No BPMN isto corresponde à decisão de possui desconto e à atividade faturar.

Receção do pagamento e entrega do equipamento

Aqui, nesta última etapa, o cliente regulariza a fatura. Após a liquidação desta estar confirmada, emite-se o recibo, entrega-se o equipamento e encerra-se o processo. Isto, no BPMN é aguardar pagamento, receber pagamento e por fim, entregar computador.

Gestão de exceções e percursos alternativos

Existem ainda algumas exceções/percursos alternativos nomeadamente na rutura de stock. Na eventualidade de não haver componentes necessários, é necessário encomendar ao fornecedor, aguardar pela mesma encomenda, receber essa encomenda e se os artigos recebidos não forem conforme os pedidos, executa-se ainda um pedido de troca ao fornecedor. Outro fluxo alternativo é ainda a recusa do orçamento por parte do cliente, que se este rejeitar a reparação ou a montagem, o processo segue diretamente para entregar o computador sem fatura nem consumo do inventário.

2.2.4 Integração do agente RPA

Na etapa 5, para evitar que uma intervenção pare por falta de componentes e tenha de seguir o percurso de exceção, o workflow n8n é executado diariamente às 9 horas. Este workflow consulta, através da camada de serviços, o stock atual e o stock mínimo de cada peça registada no Moloni ON. As peças abaixo do mínimo são agrupadas num único alerta enviado por e-mail ao gestor, que pode então encomendar ao fornecedor antes deste stock se esgotar. O workflow apenas lê os dados do ERP, não emitindo documentos nem alterando o stock.

2.2.5 Modelo de domínio e mapeamento com o Moloni ON

Na Tabela 3 é possível observar as entidades, atributos principais e a ligação ao ERP.

Tabela 3 - Entidades do modelo de domínio e mapeamento

Entidade	Atributos	Mapeamento
Cliente	MoloniCustomerId, NIF, Nome, Email, Telefone, Morada, CodigoPostal, Cidade, countryId	customers e customerCreate
Intervencao	NumeroProcesso, Tipo, Estado, DataRececao, DescricaoProblemaExigencias, RelatorioDiagnostico, ValorOrcamento, HorasTrabalho, PrecoHoraMaoDeObra, PossuiDesconto, PercentagemDesconto, TestesConcluidosComSucesso	Gestão interna
Tecnico	Nome, Email, Especialidade, Ativo	Gestão interna
CatalogoPeca	MoloniProductId, Referencia, Descricao, PrecoUnitario, StockAtual, TaxaIVA, WarehouseId	products e product, o warehouseId segue em cada linha da fatura para diminuir o stock
PecaUtilizada	Quantidade, PrecoCobrado, EstadoEncomenda, ObservacoesEncomenda	Linhas da ordem de trabalho associadas à intervenção e ao catálogo
Fatura	MoloniDocumentId, NumeroFatura, DataEmissao, DataVencimento, ValorTotalSemIVA, ValorTotalComIVA, EstadoFatura, URL_PDF, EmailEnviado	invoiceCreate, invoiceGetPDF e invoiceGetPDFToken
Recibo	MoloniReceiptId, NumeroRecibo, DataPagamento, ValorPago, MetodoPagamento	receiptCreate

3. ERPs

Este capítulo apresenta os dois sistemas ERP analisados. O Moloni ON, selecionado para o projeto e o Jasmin, estudado como alternativa. A análise de cada um incide sobre as funcionalidades utilizadas no processo, a respectiva API e as limitações encontradas.

3.1 Moloni ON

Descrição e utilização

O Moloni On é um software de gestão e faturação 100% online. Foi desenvolvido pela Moloni e está certificado pela Autoridade Tributária [13]. Destina-se, sobretudo, a micro e pequenas empresas, o que neste caso, coincide com a organização escolhida para o projeto: uma empresa pequena de serviços técnicos que vende intervenções e peças.

O Moloni On cobre as seguintes áreas: Entidades, Produtos, Armazéns e Stocks, Documentos e Configurações. Em termos de funcionalidade das entidades, temos a ficha de cliente com NIF validado, morada, contactos e condições de pagamento. No processo, isto é usado para identificar ou criar o cliente. Já os produtos referem-se a artigos de tipo de produto ou serviço, com preço, IVA, unidade e tipo. Isto é usado na mão de obra como serviço ou peças como produto. Os armazéns de stocks têm um stock mínimo e um stock por armazém. A consulta e redução de stock das peças é o que é utilizado durante o processo nas etapas 5 e 6. Os documentos são faturas, faturas de recibo, recibos, notas de crédito e orçamentos. Isto é, então, utilizado para faturas de intervenção e recibo de pagamento. Por fim, as configurações são séries de documentos, impostos, métodos de pagamento, unidades. No processo são parâmetros fixos, lidos pela camada de integração [14].

Foi então criada uma conta de teste [15] com a empresa fictícia TecAssist - ProjetoIEOP e esta empresa ficou registada na Dinamarca para não gerar dados reais em Portugal. O assistente inicial criou automaticamente uma série de documentos 2026, mas estes mesmos documentos não tinham tipo associado. Foi então necessário associar-lhe manualmente fatura, recibo,

fatura-recibo e nota de crédito. Os módulos de armazéns e stocks aparecem no menu de produtos, mas só ficam utilizáveis depois de ativados. Foram para isso criados serviços de mão de obra, três peças com stock inicial de 10 unidades e um cliente de teste, como se pode observar nas Figuras 7, 8 e 9.



Figura 5 - Lista de produtos

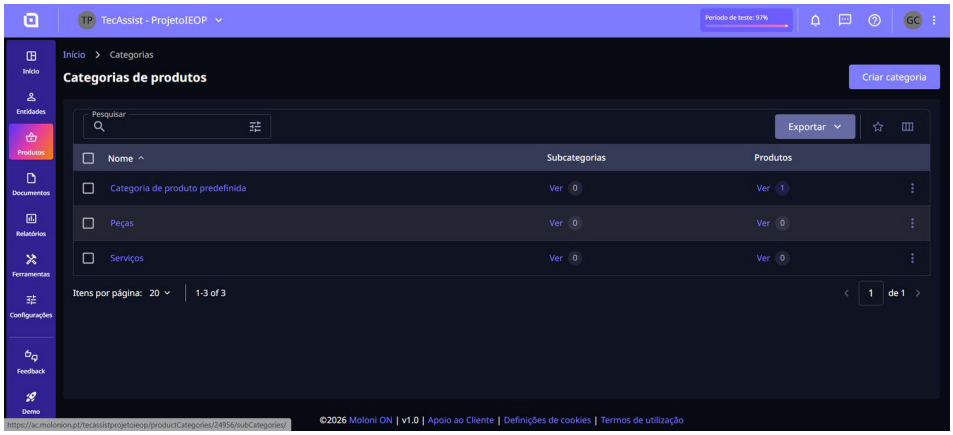


Figura 6 - Categorias de produtos

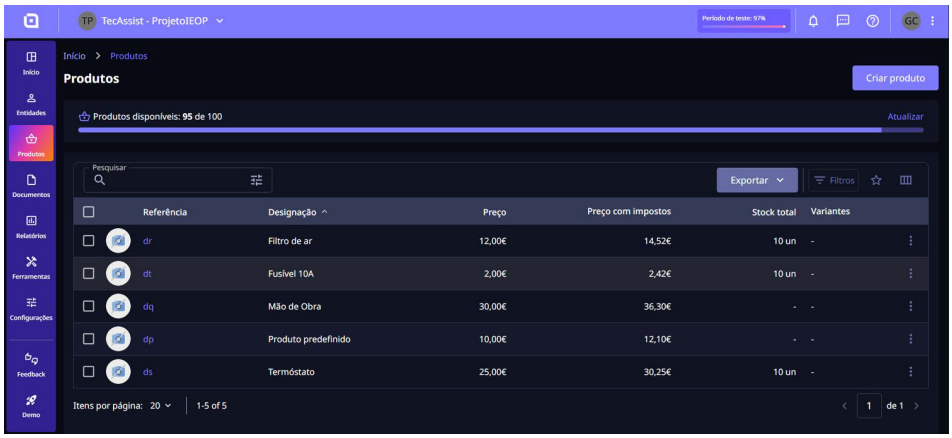


Figura 7 - Stock por produto

Foi ainda criada uma fatura rascunho com mão de obra e peças, de modo a testar se o mesmo funcionava. (Figura 10)



Figura 8 - Fatura rascunho

API

O Moloni ON disponibiliza uma API GraphQL com um único endpoint. Este dá acesso à leitura e escrita de clientes, produtos, stock e documentos. As leituras fazem-se com queries e as escritas com mutations. Na Tabela 4, estão resumidas todas as características da API GraphQL do Moloni ON.

Tabela 4 - Características da API GraphQL

Aspeto	Detalhe
Endpoint	POST https://api.molonion.pt/v1, corpo JSON com a query e as variáveis
Cabeçalhos	Content-Type: application/json e Authorization: Bearer <token>
Âmbito	Quase todas as operações recebem companyId, obtido uma vez com query me
Resposta	Cada operação devolve errors e data; errors vazio indica que houve sucesso [16]
Códigos HTTP	200 mesmo com erros de validação, 401 com falha na autenticação, 500 com erro interno
Nomes das operações	Derivam do tipo de documento: invoiceCreate, invoice, invoices, invoiceGetPDF, receiptCreate [17]

A API aceita dois métodos [18, 19], API Key e o API Client [20, 21]. Estes métodos foram comparados. (Tabela 5)

Tabela 5 - Comparação métodos

	API Key	API Client
Destino	Integrações servidor a servidor	Aplicações web que agem em nome de um utilizador
Obtenção	Conta -> API -> API Key	Conta->API-> API Client
Validade	Sem expiração ou até a data definida	Token de acesso com validade de 1 hora

Assim, depois de analisar os prós e contras de cada método, foi escolhida a API Key. A integração corre na camada de serviços sem um utilizador autorizar no browser, o que torna desnecessária a renovação de tokens de hora em hora. A chave herda as permissões do utilizador que a criou e é guardada numa variável de ambiente fora do código e do repositório. Pode-se confirmar que a mesma funciona a partir da consulta realizada. (Figura 11)



Figura 9 - Consulta me

Durante a análise e os primeiros testes com o Moloni ON surgiram algumas dificuldades, nomeadamente no estilo do GraphQL, na documentação desatualizada, módulo de acesso obrigatório, erros dentro de um http 200, validação fiscal restrita, imposto de produto não chegar à fatura, entre outros. Estas mesmas dificuldades encontram-se abaixo de escritas:

- **Estilo GraphQL** - o Mendix consome serviços de REST com facilidade, mas o GraphQL obriga a enviar a query como texto dentro de um POST JSON. Esta é uma das razões para a camada de serviços que expõe ao Mendix endpoints REST simples;
- **Documentação desatualizada** - o guia Getting Started pede alguns campos como name e slug em userCompanies, mas o schema não os inclui, falhando a query;
- **Módulo de acesso obrigatório** - apesar da Query Me ter funcionado, algumas operações da empresa devolviam um erro a dizer que a empresa não tinha acesso ativo para usar a API. Este acesso depende de um módulo da empresa que está separado da chave, tendo este mesmo sido ativado posteriormente;
- **Explorer sem persistência** – o Explorer do Moloni ON não tem persistência. Se a página for recarregada, o cabeçalho de autenticação perde-se e tem de ser colado novamente;
- **Empresa de teste fora de Portugal** - como a empresa ficou registada na Dinamarca, o assistente inicial criou impostos dinamarqueses. Tendo sido necessário criar manualmente os impostos portugueses e associá-los aos produtos.

Mais-valia e contributo para o processo de negócio

O Moloni ON acrescenta ao processo o que a aplicação Mendix não pode garantir sozinha: faturação certificada, comunicação com a Autoridade Tributária, ficheiro SAF-T, uma única ficha de clientes e o controlo de stock das peças.

Este processo utiliza 19 operações da API. As de configuração são executadas uma única vez e os resultados ficam guardados na camada de serviços. As restantes são executadas em cada pedido. Na Tabela 6 é possível observar as operações utilizadas da API do Moloni ON e a sua justificação de uso.

Tabela 6 - Operações da API do Moloni ON

Etapa	Operação	Tipo	Justificação
Configuração	me	query	Obter o companyId exigido por todas as outras operações
Configuração	documentSetsForDocument	query	Identificar a série de faturas e recibos que a API não aplica por omissão
Configuração	paymentMethods	query	Identificar os métodos de pagamento a usar no recibo
Configuração	taxes, measurementUnits, productCategories, countries	query	Parâmetros necessários para criar clientes e produtos
Configuração	productCreate	mutation	Criar o serviço "mão de obra" e as peças com stock
Pesquisa e criação de cliente no ERP	customers	query	Evitar clientes duplicados uma vez que o NIF é único [22]
Pesquisa e criação de cliente no ERP	customerNextNumber + customerCreate	query + mutation	O número de cliente é obrigatório e tem de ser único
Execução da intervenção e consulta de stock	products	query	Fornecer ao formulário do técnico a lista real de peças e preços

Execução da intervenção e consulta de stock	product	query	Mostrar o stock antes de aplicar a peça
Redução de stock e emissão de fatura	invoiceCreate	mutation	Criar a fatura com mão de obra e peças [11]
Emissão e envio da fatura	invoiceGetPDF + invoiceGetPDFToken	mutation + query	Obter o PDF que é enviado ao cliente por e-mail
Receção do pagamento e entrega do equipamento	receiptCreate	mutation	Ligar o pagamento à fatura e registar o método
Receção do pagamento e entrega do equipamento	Invoice, getCustomerRelatedDocuments	query	Consultar a fatura e os recibos associados ao cliente

Limitações

O Moloni ON é um software de faturação em gestão comercial, não um ERP completo. Não tem módulos de assistência técnica, ordens de trabalho, gestão de técnicos nem contabilidade. Por isso, os registos pedidos, a gestão de técnicos, entre outros, ficam na aplicação Mendix e no Moloni ON ficam apenas os dados comerciais e fiscais. Além disso, o plano base limita a 50 documentos por mês, o que chega para este pequeno projeto, mas não para uma empresa real. Uma limitação associada ao projeto é o próprio limite de teste será apenas um mês, embora o projeto se estenda por vários meses.

3.2 Jasmin

O Jasmin é uma solução de gestão empresarial cloud da PRIMAVERA Business Software Solutions, este está organizado em módulos funcionais [23]. A semelhança do Moloni ON destina-se a pequenas e médias empresas, mas apresenta um âmbito mais alargado do que uma aplicação de faturação, ao oferecer ainda áreas como a gestão financeira [24].

API

Ao contrário do Moloni ON, o Jasmin disponibiliza uma API REST, em que cada recurso é identificado por uma URL e manipulado com operações de criação, atualização, eliminação, consulta e listagem. Nos URLs está identificada a conta, a subscrição, o módulo funcional e o recurso pretendido.
A autenticação baseia-se no protocolo OAuth 2.0 através do serviço de identidade da PRIMAVERA, que emite os tokens de acesso necessários a cada pedido [25]. Para utilizar a API, a aplicação cliente tem de ser registada no ecossistema PRIMAVERA e obter os credenciais e as autorizações adequadas. A documentação oficial disponibiliza também uma coleção Postman para explorar e testar os pedidos. [26]

Comparação com o Moloni ON

Foi efetuada uma comparação com o Moloni ON, de modo a decidir qual seria utilizado para o projeto. Esta comparação encontra-se presente na Tabela 7.

Critério	Moloni ON	Jasmin
Tipo de API	GraphQL	REST
Autenticação	API key ou OAuth client	OAuth 2.0
Facilidade de consumo a partir do Mendix	Exige camadas de serviços	REST consumível diretamente, mas com gestão de tokens
Documentação e ferramentas	Presente	Presente
Período de teste	Trinta dias com possibilidade de extensão.	Apenas 30 dias
Âmbito funcional	Faturação, clientes, produtos, stock	Mais alargado

Apesar de parecer mais favorável, o Jasmin devido a sua API REST ser mais próximo do modelo do Mendix, foi optado pelo Moloni ON, em parte por uma autenticação mais fácil através da API Key, da facilidade de obter uma conta de teste rápida para validar as operações de leitura escrita, o âmbito do Moloni ON cobrir grande parte do processo necessário para o projeto e do facto, desde ter a possibilidade de estender os 30 dias de teste, não sendo necessário refazer parte do projeto.

Limitações

As limitações na parte do Jasmin passam muito pela dificuldade em conseguir uma conta teste plenamente funcional e na duração da mesma poder vir a ser menor e trazer mais trabalho aos membros do grupo.

4. Processo de integração

Neste capítulo está descrito como os componentes comunicam entre si, desde a integração com a Moloni ON através da camada de serviços, o estudo da alternativa do Jasmin e a automação do alerta de stock com o n8n.

4.1 Moloni ON

A integração tem como objetivo permitir a comunicação entre a aplicação de gestão de assistência técnica, desenvolvido é Mendix e o sistema que gera os dados comerciais e administrativos da empresa. O Moloni ON centraliza a informação de clientes, produtos, stock e documentos de faturação. A sua integração permite assim que a aplicação os consulte e utilize sem os duplicar em sistemas diferentes.
Para concretizar tal foi desenvolvida uma camada intermédia de serviços responsável por receber pedidos REST, comunicar com a API GraphQL do Moloni ON e devolver respostas JSON adaptadas aos restantes componentes. Assim, esta abordagem separa responsabilidades, reduz o acoplamento tecnológico e facilita a manutenção e a evolução do projeto.

4.1.1 Arquitetura e tecnologias utilizadas

A camada de integração foi desenvolvida com Node.js, JavaScript e Framework Express, que permite disponibilizar uma API REST para o consumo por aplicações externas. A comunicação entre os componentes segue o modelo seguinte [27, 28]:

- **Mendix:** interface com os utilizadores e gestão dos processos de assistência técnica;
- **Camada de Serviços:** endpoints REST, autenticação, validação de pedidos e transformação de dados;
- **Moloni ON:** ERP que disponibiliza operações de consulta e gestão de informação através de uma API GraphQL;
- **n8n:** automação que monitoriza diariamente os níveis de stock e envia alertas ao gestor.

4.1.2 Segurança e autenticação

São usados dois mecanismos distintos para manter a segurança e a autenticação, um por cada ligação entre sistemas. Entre a camada de serviços e o Moloni ON utiliza-se uma API key do ERP, enviada no cabeçalho. Os consumidores da camada de serviços têm ainda de apresentar uma chave interna do cabeçalho. Esta separação evita disponibilizar as credenciais do ERP a sistemas consumidores.

As credenciais são guardadas em variáveis de ambiente no ficheiro .env excluído do GitHub. O repositório inclui um .env.example apenas com os nomes de variáveis. É ainda verificada a autenticação com a presença e validade da chave interna, antes de permitir o acesso aos endpoints do negócio, comparando credenciais com funções criptográficas do Node.js.

Foi também implementado um mecanismo de classificação e tratamento dos erros do Moloni ON, que identifica falhas de autenticação, validação, comunicação e operação, incluindo até mesmo os erros GraphQL devolvidos no corpo de respostas de HTTP. A camada de serviços converte-os em respostas REST com código HTTP e mensagem JSON estruturada (Tabela 7). Este mesmo comportamento foi validado através de testes simulados de autenticação e validação, além de também testes independentes do módulo que construía os pedidos JSON.

Tabela 7 - Classificação de erros

Código	Origem / situação	Resposta HTTP
MOLONI_AUTENTICACAO	Falha de autenticação junto do Moloni ON, incluindo erros devolvidos dentro de HTTP 200	502
MOLONI_VALIDACAO	Dados rejeitados pela validação do Moloni ON	502
MOLONI_OPERACAO	Operação GraphQL não concluída com sucesso	502

Código	Origem / situação	Resposta HTTP
MOLONI_COMUNICACAO	Moloni ON inacessível (testado com um endereço inacessível: o servidor mantém-se ativo)	502
MOLONI_HTTP	Moloni ON devolveu um código HTTP de erro (diferente de 401/403)	502
MOLONI_RESPOSTA_INVALIDA	Resposta sem dados ou JSON inválido	502
ERRO_INTERNO	Erro inesperado na camada de serviços	500
(middleware)	Chave interna ausente / inválida / serviço indisponível	401 / 403 / 503

4.1.3 Endpoints REST implementados

Nesta primeira fase foram implementados cinco endpoints REST. Apenas o endpoint de verificação do servidor é público, os restantes estão protegidos pela autenticação interna. A consulta de produtos usa a operação GraphQL products e devolve identificadores, nomes e referências. A pesquisa de clientes usa customers, filtrando pelo campo VAT, o que permite identificar clientes já registados no ERP durante o processo. Quanto ao stock, existem consultas individuais e gerais, a consulta geral devolve os artigos com o controlo de stock ativo, com quantidades atuais e os mínimos configurados. Esta informação é ainda utilizada pelo workflow do n8n, que a consulta diariamente.

Estão previstas a implementação de 19 operações GraphQL relevantes para este projeto. A Tabela 8 indica as operações previstas e o seu estado atual.

Tabela 8 - Operações GraphQL

Operação	Estado
me	Validado
documentSetsForDocument	Validado
paymentMethods	Previsto
taxes	Validado
measurementUnits	Previsto
productCategories	Previsto
countries	Previsto
productCreate	Previsto
customers	Implementado
customerNextNumber	Previsto
customerCreate	Previsto
products	Implementado
product	Implementado
invoiceCreate	Validado
invoiceGetPDF	Previsto
invoiceGetPDFToken	Previsto
receiptCreate	Previsto
invoice	Validado
getCustomerRelatedDocuments	Previsto

Das 19 operações, 3 (customers, products e product) suportam atualmente os endpoints REST atuais. Outros cinco estão já validados, restando agora só implementá-los. As restantes integram o âmbito das fases posteriores. Algumas já foram exploradas no ERP, mas ainda não estão integradas. A implementação incremental permite assim disponibilizar primeiro as funcionalidades prioritárias.

4.1.4 Testes realizados

Para estes testes foi usada a empresa configurada no Moloni ON. Começou-se por validar a autenticação com a operação me, que confirmou o utilizador autenticado e o acesso à empresa, seguindo-se as consultas de produtos, clientes e stock confirmando a correspondência entre os dados do ERP e os resultados dos endpoints REST (Figura 12). Na consulta de produtos foram identificados cinco registos, três dos quais com controlo de stock ativo, pesquisa de clientes por NIF, avaliados nos cenários de cliente existente, cliente inexistente e ausência de parâmetro obrigatório. Foram testados identificadores de produto inválidos, produtos inexistentes e falhas de comunicação com o ERP.

```
[guilhermebarros@MacBookAir services-layer % node -e 'require("dotenv").config({quiet:true}); fetch("h]
ttp://localhost:3000/api/produtos", {headers:{"X-API-Key":process.env.INTERNAL_API_KEY}}).then(async
r=>{console.log("HTTP:",r.status);console.log("Resposta:",JSON.stringify(await r.json(),null,2))}).ca
tch(e=>console.error(e.message))'
HTTP: 200
Resposta: {
  "total": 5,
  "produtos": [
    {
      "productId": 361301,
      "name": "Produto predefinido",
      "reference": "dp"
    },
    {
      "productId": 363504,
      "name": "Mão de Obra",
      "reference": "dq"
    },
    {
      "productId": 363517,
      "name": "Filtro de ar",
      "reference": "dr"
    },
    {
      "productId": 363520,
      "name": "Termóstato",
      "reference": "ds"
    },
    {
      "productId": 363526,
      "name": "Fusível 10A",
      "reference": "dt"
    }
  ]
}
```

Figura 10 - Consulta autenticada de produtos

Foi ainda testada a criação de uma fatura em rascunho associada a um cliente de teste e com uma linha de serviço de mão de obra (Figura 13). Posteriormente foi feita uma consulta independente que confirma a existência do documento, estado, dados do cliente e os valores calculados. (Figura 14)

```
guilhermebarros@MacBookAir services-layer % node scripts/testarFatura.js --criar-rascunho
◇ injected env (5) from .env
Empresa: 4849
Série: 5832
Cliente: 420930
Estado: Rascunho
Serviço: 363504
Preço: 30
IVA (%): 23
Resposta: {
  "errors": [],
  "data": {
    "documentId": 798602,
    "number": null,
    "status": 0,
    "totalValue": 36.9,
    "grossValue": 30,
    "taxesValue": 6.9
  }
}
Rascunho criado com ID: 798602
```

Figura 11 - Criação de uma fatura rascunho

```
guilhermebarros@MacBookAir services-layer % node scripts/consultarFatura.js
◇ injected env (5) from .env
Resposta: {
  "data": {
    "invoice": {
      "errors": [],
      "data": {
        "documentId": 798602,
        "number": 1,
        "status": 0,
        "date": "2026-10-26T19:30:04.000Z",
        "expirationDate": "2026-11-25",
        "grossValue": 30,
        "taxesValue": 6.9,
        "totalValue": 36.9,
        "customer": {
          "customerId": 420930,
          "name": "goncalo costa"
        },
        "products": [
          {
            "productId": 363504,
            "name": "Mão de Obra",
            "qty": 1,
            "price": 30
          }
        ]
      }
    }
  }
}
```

Figura 12 - Consulta da fatura

Durante a exploração de faturação foram identificadas duas particularidades do serviço: o IVA tem de ser indicado explicitamente em cada linha de documento, porque os impostos associados aos produtos não garantem a aplicação de taxa pré-definida; e os rascunhos contam para o limite de documentos mensais do plano de teste, pelo que se deve evitar a criação repetida de documentos.

4.1.5 Limitações

Neste momento, a integração encontra-se apenas numa fase inicial. Estão previstos vários endpoints REST para a criação e consulta de documentos de faturação, recuperação de PDF, criação de clientes, registos de recibos e uma integração efetiva com os microflows do Mendix. A integração completa deverá ainda contemplar o registo de pagamentos e a atualização de existências associadas à faturação.

4.2 Jasmin

Durante a análise de ERPs foi considerado o Jasmin como alternativa para a integração dos processos comerciais e administrativos. O objetivo do estudo foi avaliar a possibilidade de usar a sua API para gerir clientes, artigos, existências e documentos comerciais, ligando-a à aplicação Mendix. Ao contrário do Moloni ON, o Jasmin disponibiliza uma API REST, que permite consulta, criação e atualização de recursos através de pedidos HTTP com dados em JSON. Esta análise, centrou-se nos mecanismos de comunicação, autenticação e adaptação dos serviços.

4.2.1 Abordagem tecnológica

A abordagem considerada baseava-se na web API Rest do Jasmin, cumprindo HTTPS e respostas JSON em URLs com o formato
https://my.jasminsoftware.com/api/{account}/{subscription}/{module}/{resource}
Para tal, era necessário registar uma aplicação no ecossistema PRIMAVERA, configurar a autenticação OAuth 2.0 através do serviço de identidade da PRIMAVERA e obter as

credenciais e as autorizações adequadas. Na arquitetura proposta, a camada de serviços funcionaria como um intermediário entre o Mendix e o Jasmin, gerindo os tokens, executando os pedidos REST, transformando os dados e tratando dos erros devolvidos pelo ERP. As operações pretendidas, encontram-se na Tabela 9.

Tabela 9 - Operações pretendidas

Área funcional	Operação pretendida
Clientes	Pesquisar clientes por NIF e criar fichas
Artigos	Consultar produtos, serviços e respetivos preços
Stock	Consultar existências dos artigos
Faturação	Criar e consultar documentos comerciais
Documentos	Obter representações de faturas para envio ao cliente
Pagamentos	Consultar ou registar documentos associados à liquidação

Estas operações teriam de ser mapeados para os recursos concretos da API. A disponibilidade e os parâmetros de dados de cada uma dependeriam da análise dos endpoints, apoiado depois na referência técnica e na coleção Postman. Como não foi identificada uma camada REST pré-existente que corresponde ao contrato definido pelo Mendix, o uso do Jasmin exigiria um adaptador próprio, convertendo as operações internas da camada de serviços em chamadas á API do ERP, preservando a interface REST apresentada no Mendix.

4.2.2 Decisão tecnológica

Como foi optado pelo Moloni ON como ERP principal, o desenvolvimento e a validação desta plataforma acabaram por ser reduzidas e essa escolha permitiu implementar uma camada de serviços mais funcional com autenticação, documentação e endpoints. Assim, o Jasmin permaneceu como alternativa estudada, porém não escolhida.

5. Automação com n8n

Para a automação inicial com o n8n, foi escolhido o alerta de stock baixo de peças. Este workflow é executado automaticamente uma vez por dia, consultando o stock atual e o stock mínimo de cada peça no Moloni ON. As peças abaixo do mínimo são reunidas num único alerta, enviado por e-mail ao gestor, que pode assim encomendar ao fornecedor antes do stock esgotar.

Esta automatização liga-se à etapa 5 e ao percurso da exceção: encomendar ao fornecedor e aguardar a encomenda. Isto ocorre quando o técnico precisa de uma peça inexistente em armazém e atrasa assim a reparação ou a montagem. O alerta antecipa a encomenda e reduz o número de vezes em que o processo segue este percurso.

5.1 Decisão tecnológica

No processo atual, o controlo de stock dependeria de alguém se lembrar de verificar as quantidades. A falta de uma peça só é descoberta quando o técnico precisa dela, o que obriga a encomendar com a intervenção já em curso e atrasa a entrega ao cliente. A automatização traz assim:

- prevenção em vez de reação, uma vez, que a encomenda é feita antes da peça faltar;
- zero trabalho manual, uma vez que a verificação ocorre sozinha, todos os dias, sem ninguém ter que abrir o Moloni ON;
- um único aviso consolidado, uma vez que o gestor recebe um e-mail com a lista de todas as peças em falta e não um por peça.

5.2 Escolha da ferramenta

Para escolher a ferramenta de automação, foi feita uma comparação entre cinco ferramentas, como se pode observar na Tabela 10. Após isso foi escolhido o n8n por o mesmo aparentar ter mais benefícios e facilidade no uso [29, 30, 31, 32, 33, 34, 35].

Tabela 10 - Comparação das ferramentas

Critério	n8n	Make	Zapier	UiPath	UI.Vision
Tipo	Workflows por API	Workflows por API	Workflows por API	RPA (interface)	RPA (browser)
Onde corre	Local (self-hosted) ou cloud	Apenas cloud	Apenas cloud	Local (Windows)	Extensão do browser
Custo	Gratuito em self-hosted	Plano gratuito com limites	Plano gratuito com limites	Edição Community gratuita	Gratuito
Adequado havendo API	Sim	Sim	Sim	Pouco indicado	Pouco indicado

5.3 Descrição do workflow

Observando a Figura 15, é possível reparar que após um gatilho é efetuado um pedido HTTP, que por sua vez lista os produtos e verifica se os mesmos estão abaixo de stock ou não, sendo por fim enviado um e-mail para o gestor. Isto encontra-se mais aprofundado na Tabela 11.

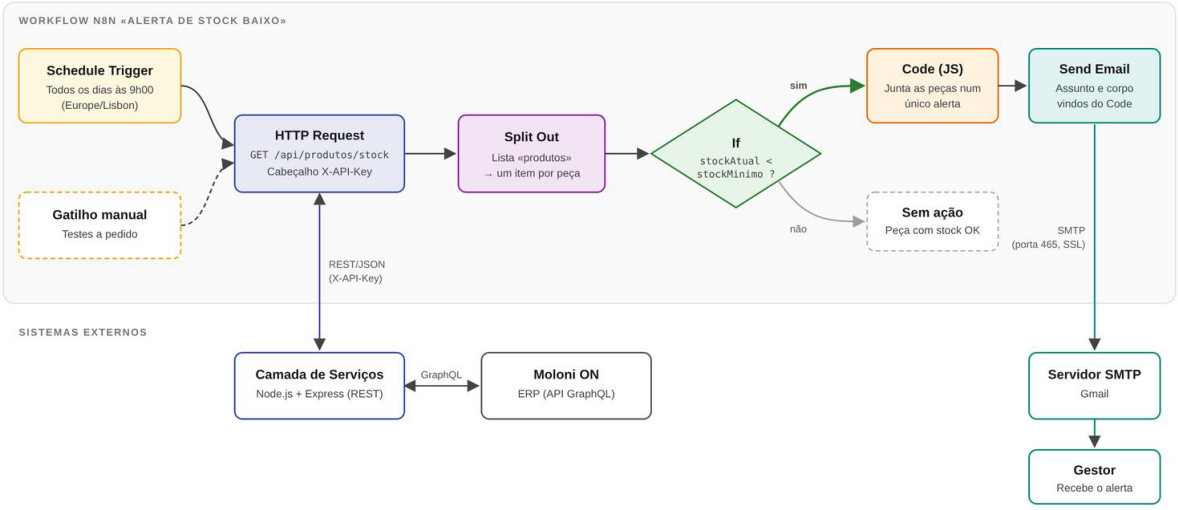


Figura 13 - Diagrama do workflow

Tabela 11 - Descrição do Workflow

Evento	Tipo	Função
Schedule Trigger	Gatilho	Inicia o workflow todos os dias às 9h. O fuso horário (Europe/Lisbon) foi definido nas definições do workflow, porque o n8n usa America/New_York por omissão. O gatilho manual usado no desenvolvimento ficou ligado ao mesmo nó, para testes a pedido.
HTTP Request	Integração	Pede à camada de serviços a lista de peças com stock. Autentica-se com o cabeçalho X-API-Key, guardado como credencial encriptada do n8n (Header Auth).
Split Out	Transformação	A resposta chega num único item com a lista produtos; este nó transforma-a num item por peça, para ser avaliada individualmente.
If	Decisão	Compara stockAtual com stockMinimo. As peças abaixo do mínimo seguem pelo ramo true; as restantes pelo ramo false, que termina sem ação.
Code (JavaScript)	Transformação	Junta todas as peças do ramo true num único item com o assunto e o corpo do e-mail.
Send Email	Notificação	Envia o alerta ao Gestor por SMTP, em texto simples; o assunto e o corpo vêm do nó anterior. Os dados da conta de e-mail ficam guardados como credencial encriptada (SMTP).

5.4 Testes

Nas Figuras 16, 17, 18, 19, 20 e 21, é possível observar todo o processo de criação do workflow, bem como os testes. Para este mesmo efeito foi criada uma conta de e-mail para o projeto que faz de gestor e é utilizada como remetente e destinatária. O envio usa o servidor SMTP do Gmail com a palavra passe da aplicação, guardada como credencial encriptada.

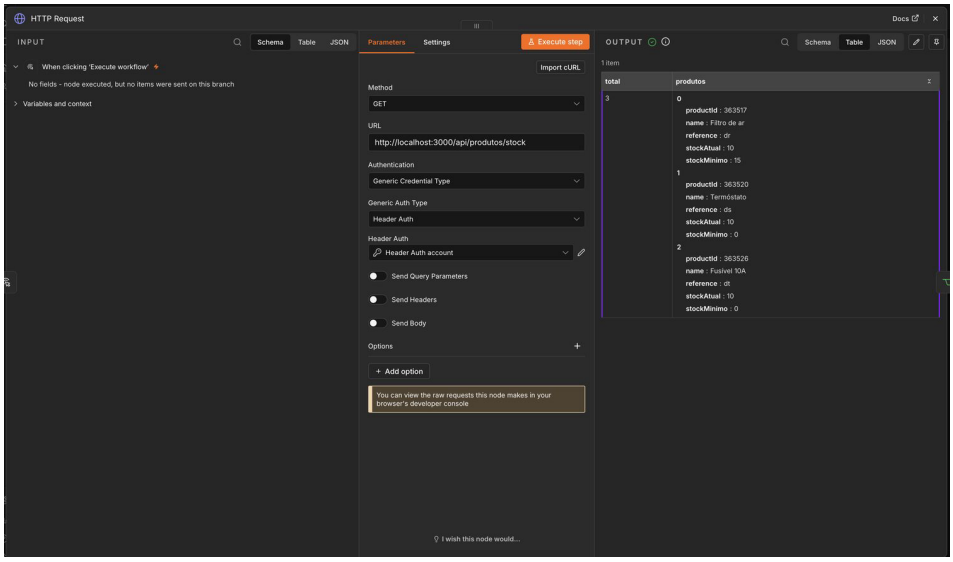


Figura 14 - HTTP request

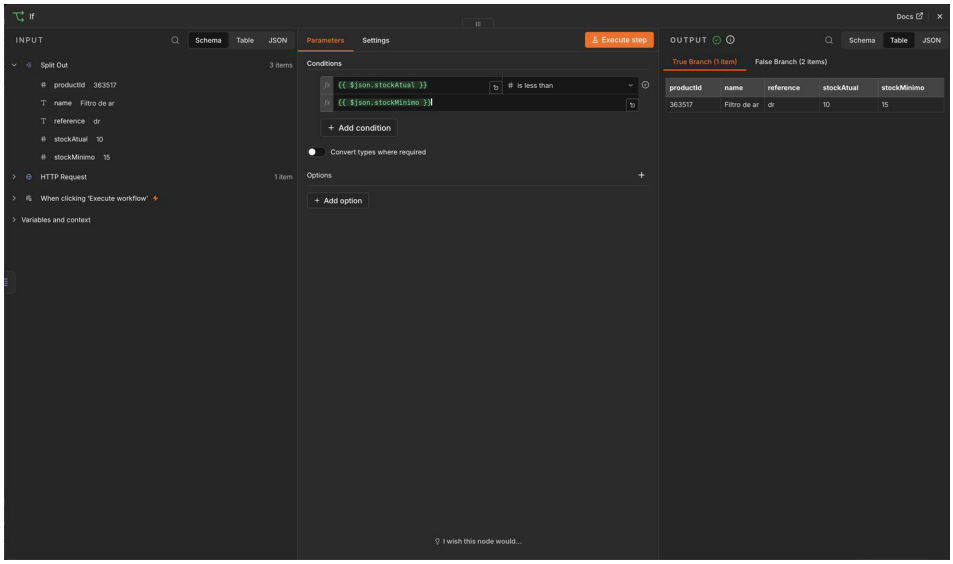


Figura 15 - if

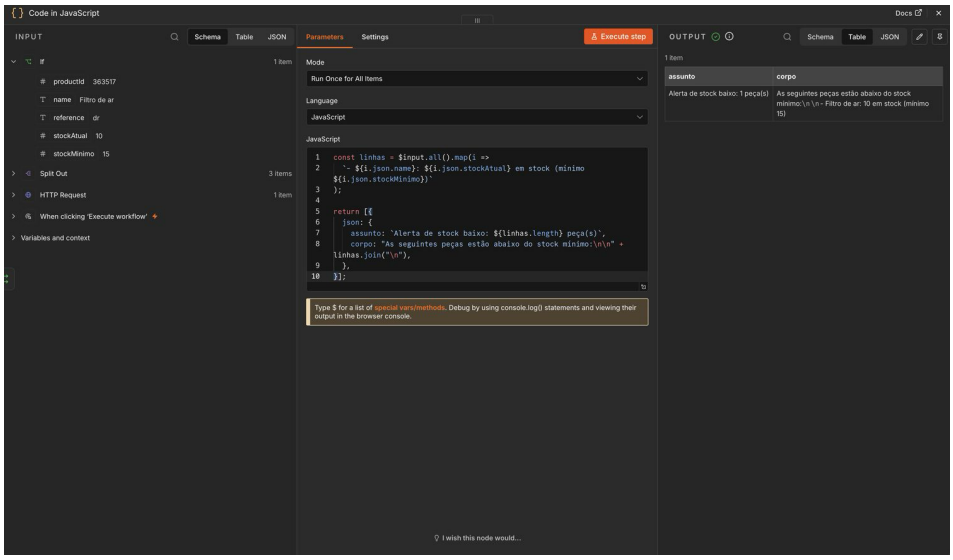


Figura 16 - Mensagem de alerta



Figura 17 - Email de alerta

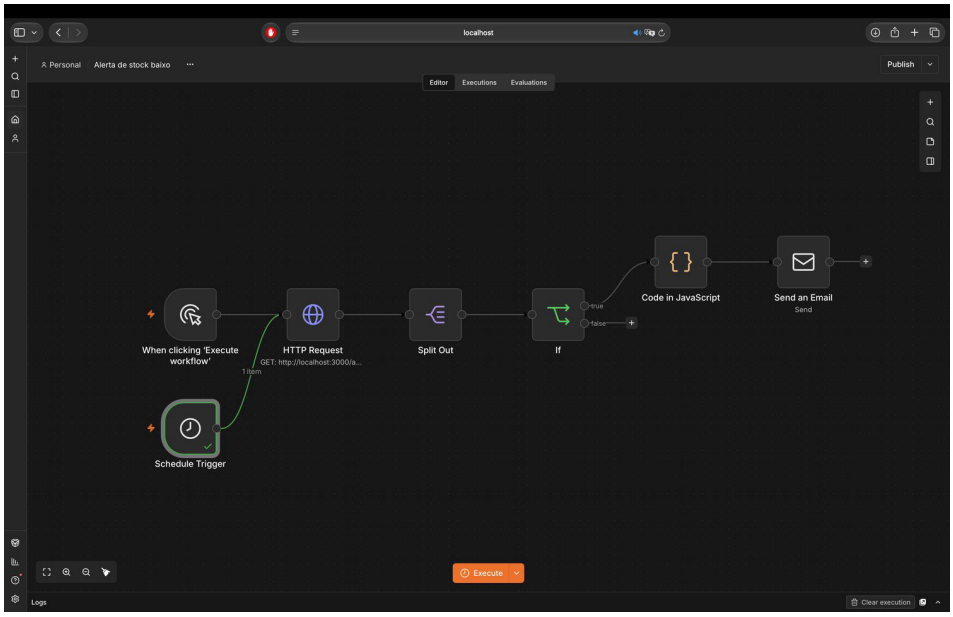


Figura 18 - Workflow final

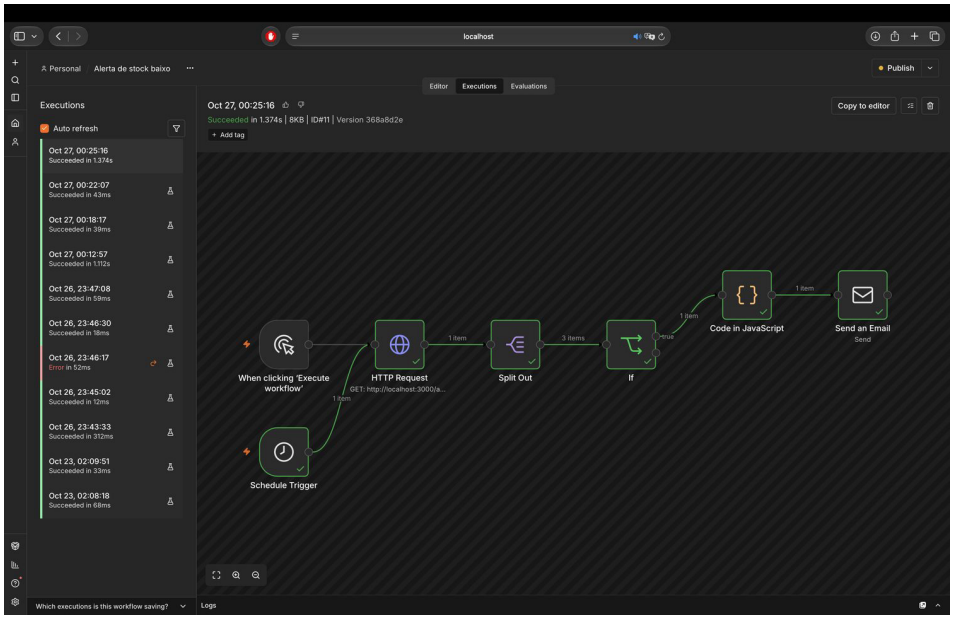


Figura 19 - Execução lançada pelo agendamento

5.5 Limitações e trabalho futuro

Surgem como limitações a dependência de stock mínimo estar configurada em cada peça de Moloni ON, por exemplo, se uma peça tiver um mínimo de zero, nunca é gerado um alerta. Ainda sofre uma dependência da camada de serviços, se a mesma estiver em baixo, o pedido falha e o alerta desse dia não é enviado. Podem ainda surgir alertas repetidos, caso a peça não for reposta.

Como o trabalho futuro, estão previstas a implementação de mais algumas automações, nomeadamente: envio faturas por e-mail e acompanhamento de pagamentos.

6. Implementação

*[Obrigatório para entrega **final**]*

6.1 Modelação do processo

6.2 Modelo de dados

6.3 Interfaces com o utilizador/Formulários

6.4 Regras de negócio

6.5 Performers

6.6 Integração

6.7 Execução do projeto

7. Conclusões

*[Obrigatório para entrega **final**]*

7.1 Objetivos

7.2 Resultados alcançados

7.3 Dificuldades e limitações

8. Referências

[1] Sanchis, R.; García-Perales, Ó.; Fraile, F.; Poler, R. Low-Code as Enabler of Digital Transformation in Manufacturing Industry. Applied Sciences, 10(1), 12, 2020. https://www.mdpi.com/2076-3417/10/1/12 (acedido a 28/10/2026).

[2] van der Aalst, W. M. P.; Bichler, M.; Heinzl, A. Robotic Process Automation. Business & Information Systems Engineering, 60(4), 2018. https://aisel.aisnet.org/bise/vol60/iss4/1 (acedido a 28/10/2026).

[3] Mendix. Team Server. Mendix Documentation. https://docs.mendix.com/developerportal/repository/team-server/ (acedido a 28/10/2026).

[4] Mendix. Consumed REST Service. Mendix Documentation. https://docs.mendix.com/refguide/consumed-rest-services/ (acedido a 28/10/2026).

[5] Mendix. Use the Call REST Service Action in a Microflow. Mendix Documentation. https://docs.mendix.com/refguide/integration/use-call-rest-action-in-microflow/ (acedido a 28/10/2026).

[6] OpenJS Foundation. Express – Routing. https://expressjs.com/en/guide/routing (acedido a 28/10/2026).

[7] GraphQL Foundation. GraphQL Specification. https://spec.graphql.org/draft/ (acedido a 28/10/2026).

[8] Moloni ON. Moloni ON API – Getting Started. https://docs.molonion.pt/guides/gettingStarted (acedido a 20/10/2026).

[9] Mendix. Email Connector. Mendix Documentation. https://docs.mendix.com/appstore/modules/email-connector/ (acedido a 28/10/2026).

[10] Object Management Group. Business Process Model and Notation (BPMN), versão 2.0. https://www.omg.org/spec/BPMN/ (acedido a 28/10/2026). Normalizada como ISO/IEC 19510:2013.

[11] Moloni ON. Creating an Invoice. https://docs.molonion.pt/guides/documents/creatingAnInvoice (acedido a 20/10/2026).

[12] Moloni ON. Downloading a Document PDF. https://docs.molonion.pt/guides/documents/downloadingDocumentPDF (acedido a 20/10/2026).

[13] Moloni ON. Moloni ON – Da faturação à gestão integrada. https://www.molonion.pt/ (acedido a 20/10/2026).

[14] Moloni ON. Módulos e preços. https://www.molonion.pt/pricing (acedido a 20/10/2026).

[15] Moloni ON. First Time Invoicing. https://docs.molonion.pt/guides/gettingStarted/firstTimeInvoicing (acedido a 20/10/2026).

[16] Moloni ON. Error Handling. https://docs.molonion.pt/guides/advanced/errorHandling (acedido a 20/10/2026).

[17] Moloni ON. Document Types. https://docs.molonion.pt/guides/documents/documentTypes (acedido a 20/10/2026).

[18] Moloni ON. API Keys. https://docs.molonion.pt/guides/gettingStarted/apiKeys (acedido a 20/10/2026).

[19] Moloni ON. API Clients. https://docs.molonion.pt/guides/gettingStarted/apiClients (acedido a 20/10/2026).

[20] Hardt, D. (ed.). The OAuth 2.0 Authorization Framework. RFC 6749, IETF, 2012. https://datatracker.ietf.org/doc/html/rfc6749 (acedido a 28/10/2026).

[21] Jones, M.; Hardt, D. The OAuth 2.0 Authorization Framework: Bearer Token Usage. RFC 6750, IETF, 2012. https://datatracker.ietf.org/doc/html/rfc6750 (acedido a 28/10/2026).

[22] Moloni ON. Customers. https://docs.molonion.pt/guides/entities/customers (acedido a 20/10/2026).

[23] PRIMAVERA BSS. Jasmin Developers – Web API Reference. https://jasminsoftware.github.io/ (acedido a 28/10/2026).

[24] PRIMAVERA BSS. Jasmin – PRIMAVERA Developers Network. https://developers.primaverabss.com/en/jasmin/ (acedido a 28/10/2026).

[25] PRIMAVERA BSS. How does OAuth 2.0 authorization work? PRIMAVERA Developers Network.

https://developers.primaverabss.com/en/jasmin/resources/references/article/how-does-oauth-2-0-authorization-work/ (acedido a 28/10/2026).

[26] PRIMAVERA BSS. Jasmin API Collection. Postman. https://www.postman.com/primaverabss-developers/primaverabss-jasmin/documentation/c8ejbr3/jasmin-api-collection (acedido a 28/10/2026).

[27] OpenAPI Initiative. OpenAPI Specification v3.0.3. https://spec.openapis.org/oas/v3.0.3.html (acedido a 28/10/2026).

[28] Redocly. Redocly CLI – lint command. https://redocly.com/docs/cli/commands/lint (acedido a 28/10/2026).

[29] UiPath. UiPath Platform. https://www.uipath.com/ (acedido a 26/10/2026).

[30] UI.Vision. UI.Vision RPA. https://ui.vision/ (acedido a 26/10/2026).

[31] n8n. n8n Docs. https://docs.n8n.io/ (acedido a 26/10/2026).

[32] Make. Pricing. https://www.make.com/en/pricing (acedido a 26/10/2026).

[33] Zapier. Pricing. https://zapier.com/pricing (acedido a 26/10/2026).

[34] n8n. Built-in integrations (HTTP Request, Split Out, If, Code, Schedule Trigger e Send Email). https://docs.n8n.io/integrations/builtin/ (acedido a 26/10/2026).

[35] n8n. Sustainable Use License. https://docs.n8n.io/n8n-community-license/sustainable-use-license (acedido a 26/10/2026).

9. Anexos

9.1 Gestão de Projeto

Equipa e Responsabilidades

O desenvolvimento do projeto assenta numa abordagem colaborativa com distribuição de responsabilidades pelos membros do grupo, de acordo com as principais componentes tecnológicas do projeto. Esta distribuição permitiu assim desenvolver os componentes de forma relativamente independente, mantendo a coordenação nos pontos de integração, em particular em operações que há vários componentes, como consulta de cliente, gestão de peças, faturação e acompanhamento dos pagamentos.

Tabela 12 - Grupo e áreas de responsabilidade

Número	Nome	Função	Responsabilidades
22423	Mátio Macedo	Gestor do Projeto	Planeamento de atividades, acompanhamento da execução, organização dos entregáveis e articulação entre todos os elementos do grupo
36701	Gonçalo Costa	Analista ERP	Análise, configuração e avaliação do Moloni ON e estudo do Jasmin como alternativa
33366	Gonçalo Rodrigues	Analista Low Code / DPA	Aplicação Mendix: interfaces, modelo de domínio, microflows e processos de negócio, descrição do processo
31419	Guilherme Barros	Analista de Integração	Camada de serviços em Node.js e Express: comunicação

			REST/GraphQL, autenticação, validação e tratamento de erros, arquitetura
31434	Mário Ferreira	Analista RPA	Workflows n8n: monitorização do stock e alertas por e-mail

Diagrama de Gantt

Na Figura 22, é possível visualizar o planeamento da fase de desenvolvimento, desde a primeira semana de novembro até a entrega do relatório e à apresentação. Os períodos de 21 de dezembro a 3 de janeiro correspondem a férias, pelo que nessa fase se prevê apenas a redação das secções de implementação.

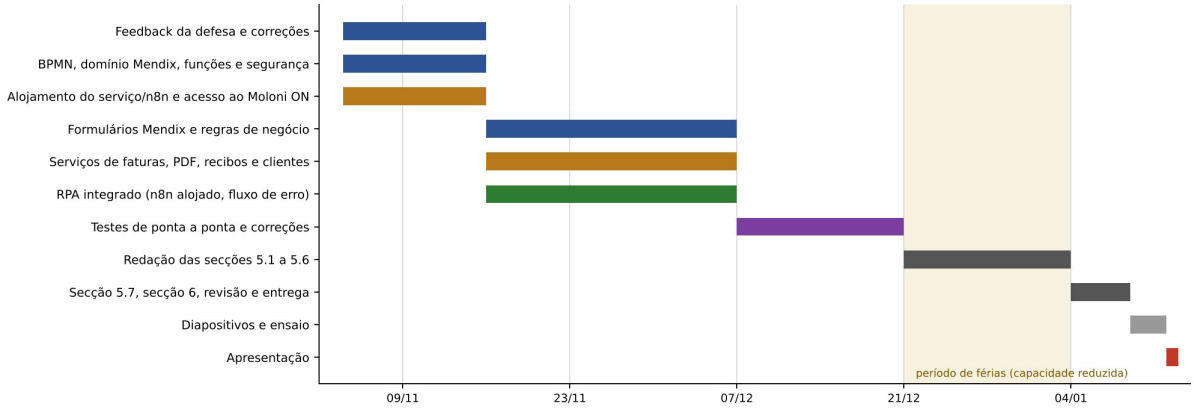


Figura 20 - Diagrama Gantt

Ferramentas de comunicação inter-equipa

Foi criado um servidor no Discord para envio de mensagens e partilha de chaves privadas. Além disso, para acompanhamento e reuniões, foi criado um grupo no Teams de modo a fazer reuniões não presenciais quando necessário.

Metodologia colaborativa de desenvolvimento

Encontra-se a ser utilizada uma metodologia scrum como forma colaborativa do desenvolvimento. Estão previstos sprints de uma/duas semanas para a implementação do projeto em si.
Foi ainda criado um repositório no GitHub onde está a ser armazenado todo o conteúdo elaborado no projeto, desde código a documentação. Serão ainda criados issues no GitHub Projects de modo a dividir as tarefas da implementação entre os membros do grupo.

Diário do projeto

Semana 1 (12 a 18 de outubro)

Tarefa	**Mátio Macedo**	**Gonçalo Costa**	**Gonçalo Rodrigues**	**Guilherme Barros**	**Mário Ferreira**
Reunião de equipa (escolha do tema)	1h	1h	1h	1h	1h
Distribuição das tarefas	1h30				
Criação da conta de developer do Jasmin e de uma app no Primavera Identity		2h			

Análise da Web API do Jasmin: estrutura dos endpoints, autenticação OAuth2, coleção Postman		1h			
Tentativa de autenticação e de primeiro pedido à API do Jasmin		1h			
Total da semana	2h30	5h	1h	1h	1h

Semana 2 (19 a 25 de outubro)

Tarefa	**Mátio Macedo**	**Gonçalo Costa**	**Gonçalo Rodrigues**	**Guilherme Barros**	**Mário Ferreira**
Reunião de equipa	1h	1h	1h	1h	1h
Distribuição das tarefas	1h30				

Análise do trabalho realizado	1h30				
Análise da API do Moloni ON (estrutura, autenticação, operações) e escolha das operações para o processo		2h			
Criação da conta Moloni ON e da empresa de teste; ticket ao apoio da Moloni		1h			
Configuração da empresa de teste: chave de API, séries de documentos, armazém, categorias, produtos e cliente		1h30			
Testes iniciais no Explorer (query me,		1h			

IDs) e teste de stock com fatura em rascunho					
Redação das tabelas de dados ERP		1h30			
Escolha da tarefa a automatizar (alerta de stock baixo)					1h
Instalação e exploração do n8n; workflow de teste com dados fictícios (gatilho, Code, If)					2h
Comparação das ferramentas de automação (n8n, Make, Zapier, UiPath, UI.Vision) e justificação da escolha do n8n					0h45
Análise dos requisitos de				0h45	

integração e definição da arquitetura da Services Layer (Node.js, Express e Moloni ON)					
Configuração inicial da Services Layer, variáveis de ambiente e autenticação por API Key				0h45	
Implementação e testes dos endpoints REST de consulta de produtos e clientes				1h30	
Implementação dos endpoints de stock e integração com as operações GraphQL do Moloni ON				1h15	
Elaboração e validação da				0h45	

especificação OpenAPI dos serviços REST					
Total da semana	4h	8h	1h	6h	4h45

Semana 3 (26 a 1 de novembro)

Tarefa	**Mátio Macedo**	**Gonçalo Costa**	**Gonçalo Rodrigues**	**Guilherme Barros**	**Mário Ferreira**
Reunião de equipa	1h	1h	1h	1h	1h
Distribuição das tarefas	1h30				
Análise do trabalho realizado	1h30				
Redação do relatório	8h				
Criação do PowerPoint	2h				
Testes da API no Explorer: ativação do módulo, impostos, invoiceUpdate com IVA 23%, PDF		1h30			
Correção dos impostos (IVA português) e associação aos produtos		0h30			
Atualização de figuras e dificuldades		1h			
Guia de ligação do Mendix ao Moloni ON		1h			

Integração do n8n com a camada de serviços (endpoint de stock) e testes com dados reais do Moloni ON					1h45
Documentação RPA; envio de e-mail (SMTP) e agendamento diário do workflow n8n: testes e exportação					1h15
Diagrama do workflow, revisão da documentação RPA					1h05
Testes de integração com o Moloni ON e prova de conceito de criação e consulta de fatura em rascunho				1h	
Implementação e validação do tratamento de erros GraphQL e HTTP na Services Layer				1h	
Elaboração da documentação para o relatório intermédio				2h	
Criação do diagrama de arquitetura, organização das capturas de ecrã e				1h	

publicação da documentação no GitHub					
Atualização da documentação técnica				0h30	
Preparação do servidor Express e publicação da Services Layer no Render				1h	
Testes dos cinco endpoints REST, autenticação e comunicação com o Moloni ON através do Render				0h45	
Análise do processo de assistência técnica e redação dos intervenientes, 8 etapas, exceções e mapeamento com os diagramas BPMN e o Moloni ON)			2h30		
Revisão da secção 2.2 e alinhamento técnico com a equipa			1h30		
Descrição do componente Mendix			0h45		

Segurança no Mendix: papéis de módulo e de utilizador			1h30		
Páginas de gestão das entidades			1h30		
Personalização da página de login e do tema			3h		
Página inicial do Gestor com faturas por pagar e peças com stock baixo			0h45		
Diagnóstico do envio de e-mail pelo Mailtrap			0h30		
Camada de serviços a correr localmente: clone do repositório, ficheiro .env, arranque, teste de /api/health e importação do contrato OpenAPI no Mendix com constantes de URL e API Key			1h30		
Microflow Atualizar_Catalogo: chamada a /api/produtos/stock, estrutura de resposta, ciclo sobre as peças, criação ou atualização			2h		

do CatalogoPeca, tratamento de erro e teste com a camada desligada					
Formulário de registo do pedido (Intervencao_NewEdit) com cliente, tipo, descrição e número de processo automático			0h45		
Página do Técnico e microflows de estado			2h30		
Gestão das peças utilizadas na intervenção			2h		
Total da semana	14h	5h	19h15	8h15	5h05

Reuniões de equipa

Reunião 1

Data: 13/10/2026
Local: L3.4 - ESTG UNIPVC
Presenças: Todos os membros
Objetivos: Decisão do tema e entrega das primeiras tarefas
Descrição/Conclusões: Tema decidido e tarefas entregues ao Gonçalo Costa.

Reunião 2

Data: 20/10/2026
Local: L3.4 - ESTG UNIPVC
Presenças: Todos os membros
Objetivos: Ponto de situação e distribuição de tarefas
Descrição/Conclusões: Início do desenvolvimento por parte de todos os membros após decisão de usar o Moloni ON.

Reunião 3

Data: 27/10/2026
Local: L3.4 - ESTG UNIPVC
Presenças: Todos os membros
Objetivos: Ponto de situação, distribuição de tarefas e acompanhamento
Descrição/Conclusões: Aspetos a melhorar, preparar diagrama de Gantt e preparação entrega intermédia.